WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

# Against KAPE and EZ

# WAC (Windows Artefacts Collector)

## Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

# WAC — Windows Artefacts Collector

Against KAPE and Eric Zimmerman's tools


| | |
|---|---|
| Software version | 1.4.0 |
| Document edition | October 2026 |
| Publisher | Centre d'Excellence Cyberdéfense Aérospatiale (CEC) |
| | École de l'air et de l'espace, air base 701, Salon-de-Provence |



This document confronts WAC with the community's reference tools: KAPE for collection, Eric Zimmerman's command-line tools for decoding. On the same exhibits of the same machine, it compares running times, what each one takes, the accuracy of the values — every difference settled by an independent reference —, the output formats, and the traces each one leaves on the examined machine. It says what the confrontation taught WAC, and what it still holds against it.

# Contents


**1 The method** **2**

1.1 What is compared . . . . . 2

1.2 Who is right when they differ . . . . . 2

1.3 The test bench . . . . . 3

**2 Summary** **4**

**3 Running times** **6**

3.1 Decoding . . . . . 6

3.2 Collection . . . . . 6

3.3 Collection from one same state . . . . . 7

**4 What each one takes** **8**

4.1 The same exhibits, different files . . . . . 8

4.2 What is recorded of each exhibit . . . . . 8

**5 Traces left on the examined machine** **10**

5.1 The measure . . . . . 10

5.2 What each one leaves . . . . . 10

5.3 What the measure fixed in WAC . . . . . 12

**6 Accuracy** **13**

6.1 In favour of WAC . . . . . 13

6.2 In favour of the EZ tools . . . . . 14

**7 Output formats** **15**

7.1 What each tool writes . . . . . 15

7.2 What sets them apart . . . . . 15

**8 What the confrontation taught WAC** **17**

**9 Limits of the comparison** **18**

CHAPTER 1

# The method

## 1.1 What is compared

Two questions, two comparisons.

**Collection: WAC against KAPE**
Both tools collect the same machine one after the other; what each copies is compared, the time it takes, and the SHA-1 of the files both copy.

**Traces left: WAC, KAPE and the EZ tools**
Each one is started from the same state of the same machine, under Sysmon; what it writes there is recorded, what Windows writes there because of it, and what a monitoring tool sees of it (chapter 5).

**Decoding: WAC against the EZ tools**
On the *same exhibits* — the exhibit store of a WAC collection —, WAC converts an artefact and the matching Eric Zimmerman tool decodes it; both outputs are compared record by record and field by field.

## 1.2 Who is right when they differ

A difference between two tools does not say which one is wrong. Every difference was therefore settled by a third source, independent of both:

- Windows itself: the XML it renders for an event (`Get-WinEvent`), a file's date, the NTFS object identifier (`fsutil objectid`), the live registry, a file's hash (`Get-FileHash`);
- another implementation: `dissect.esedb` (Fox-IT) for ESE databases;
- the format itself: sequence numbers and Marvin32 digests of the hive logs, read byte by byte;
- two artefacts of the same machine cross-checked: Prefetch for execution, a row's timestamp for its other dates.

Each difference is then classed: *defect of WAC* (fixed), *error of the EZ tool* (checked), or *convention* of either — two forms of one value.

## 1.3 The test bench

| | |
|---|---|
| Machine | Windows 11 Pro (build 26200), virtual machine |
| EZ tools | version 2026.5.0 (.NET 9.0.20) |
| KAPE | version 1.3.0.2, target `KapeTriage` |
| WAC | version 1.4.0 with the fixes this confrontation brought |
| Traces measure | clone of the machine, no network, frozen; Sysmon 15.22; tools on a USB key (K:), one run per tool |

Everything can be replayed: `vmtest/ez-compare.sh <artefact>` times WAC and the EZ tool on the same exhibits then compares their outputs; `vmtest/kape-compare.sh` does the same for collection; `vmtest/footprint/run-footprint.sh` replays the measure of the traces and `footprint-diff.py` analyses it. The EZ tools ran with their full date precision (`--mp`, or `--dt` with seven decimals where `--mp` is not enough), so as not to blame them for a loss of precision a setting avoids.

CHAPTER 2

# Summary

| Artefact | Volume | Result | Errors of the EZ tool, checked |
|---|---|---|---|
| $MFT — MFTECmd | 362,081 records | paths, states, 8 dates identical | sizes of 1,490 WOF files and 3 open logs: Windows gives WAC's |
| EVTX — EvtxECmd | 190,613 events | system fields identical; values explained | 6 wrong values (Windows); 335 Perflib events unreadable; cleared logs ignored |
| Prefetch — PECmd | 379 files | all identical | hash without its leading zero (19) |
| Shortcuts — LECmd | 3 files | identical to 100 ns | — |
| Jump lists — JLECmd | 13 shortcuts | identical to 100 ns | — |
| Shellbags — SBECmd | 27 folders | identical | wrong NodeSlot at first level (7); HasExplored always false |
| Amcache — AmcacheParser | 1,258 entries | 124 columns identical | 2,340 invented values; partial SHA-1 not flagged |
| ShimCache — AppCompat-CacheParser | 232 entries | identical | misleading execution flag |
| Recycle bin — RBCmd | 19 files | identical | — |
| SRUM — SrumECmd | 60,804 rows | identical | 17 wrong energy rows (dissect) |
| Registry — RECmd | 9 artefacts | identical | 4 hives rolled back to an older state |
| Collection — KAPE | 1,173 common files | SHA-1 identical | — |

**Key point**

On everything both sides read, WAC gives the same values as the reference tools, or more accurate ones, each time established by an independent source. The confrontation also revealed 24 defects of WAC, all fixed (chapter 8), and coverage gaps, opened for follow-up (section 6.2).

Started from a key, WAC writes nothing on the examined machine: no file, no registry value, no event — only the traces Windows leaves of any program started remain (Amcache, ShimCache). KAPE and the EZ tools, .NET programs, write to the user's registry and, for KAPE's modules, files on the system volume (chapter 5).

CHAPTER 3

# Running times

## 3.1 Decoding

Same machine, same exhibits; each tool handles its artefact only.

| Artefact | WAC | EZ tool | What WAC does besides |
|---|---|---|---|
| $MFT (380 MB) | 7.3 s | 8.1 s | two volumes (C: and G:) against one |
| EVTX (305 logs) | 16.8 s | 22.0 s | plain-text message rebuilt for every event |
| Registry (Kroll batch) | 2.0 s | 7.6 to 13.6 s | every hive replayed, every registry artefact |
| SRUM (21 MB) | 3.2 s | 2.5 s | 2 more tables, identities resolved |
| Prefetch (380) | 3.6 s | 0.7 s | dated local time, paths with drive letter |
| Shortcuts, jump lists | 1.8 s | 0.3 s | |
| Recycle bin | 1.9 s | 0.5 s | |
| Amcache, ShimCache, shellbags | 2.0 s | 0.5 to 1.4 s | one pass for all three (registry key) |

On the large artefacts, WAC is faster. On the small ones, the EZ tools are: every WAC conversion pays a fixed cost of 1.5 to 3.4 s, measured with a trivial artefact. That cost is the verified working copy of the hives, the replay of their logs, and the reading of the suspect's time zone and profiles. It is what lets WAC date every local time with the offset in force at that date, and give the drive letter; the EZ tools give neither.

## 3.2 Collection

| Tool, setting | Time | Exhibits | Volume |
|---|---|---|---|
| KAPE, `KapeTriage` | 58 s | 1,047 | 0.97 GB |
| WAC, kit, no binaries | 47 s | 1,898 | 2.54 GB |
| WAC, kit, no binaries, before B-033 | 212 s | 2,622 | 1.69 GB |

| Tool, setting | Time | Exhibits | Volume |
|---|---|---|---|
| WAC, full kit (`binary: unverified`), before B-033 | 454 s | 61,294 | 3.6 GB |

The first two rows are measured in the same run, one after the other. On a comparable scope, WAC now collects faster than KAPE, bringing back more than twice as much data. It was five times slower at first (item B-033):

- each raw reading opened its volume again and walked its paths from the root; the volumes are now opened once, with the cache of their directories;
- the collection took the files of the 1,426 declared event providers (739 MB, 61 s); it now takes only those of the providers the collected logs cite (324 on the VM, 178 MB, 15 s), which is enough to rebuild every event's text.

The full kit also checks the Authenticode signature of every executable on the machine and copies those whose signature does not hold, which KAPE does not do.

## 3.3 Collection from one same state

Measures of the traces comparison (chapter 5): every tool starts from the same frozen state of the machine, started from the key; the duration is that of the task that starts it, in brackets the one the tool reports.

| Tool, setting | Duration | Taken | On the key |
|---|---|---|---|
| KAPE, `!SANS_Triage` | 127 s (115 s) | 2,291 files | 2,286 files, 1.25 GB |
| KAPE, `!SANS_Triage` and `!EZParser` modules | 235 s (223 s) | 2,288 files, 14 tools | 2,345 files, 1.63 GB |
| EZ tools on the live machine (6 tools) | 132 s | $MFT, logs, hives, Prefetch, Amcache, ShimCache | 36 CSV, 0.55 GB |
| WAC, collection (default setting) | 592 s (588 s) | 11,830 exhibits, 67,882 recorded | 8,240 files, 4.9 GB |

WAC's collection is the longest because it reads *every* executable of the machine: the conversion will happen elsewhere, and it does not know yet which ones the artefacts will cite. It computes their fingerprints, checks their signature and copies only those whose signature does not hold; hence 67,882 exhibits recorded for 11,830 copied. The conversion, on the analysis workstation, reads nothing of the machine (chapter 5). KAPE copies the files of its targets without examining them.

CHAPTER 4

# What each one takes

## 4.1 The same exhibits, different files

Same runs as the previous section: WAC's collection, KAPE with `!SANS_Triage`, compared by the original path of every exhibit (WAC's manifest, KAPE's copy log).

| Exhibits | Number |
|---|---|
| Taken by both | 1,151, 1,011 of them with the same SHA-1 |
| Taken by KAPE alone | 1,140: the user's profile (930, browsers and applications), the NetworkService profile (91), `SleepStudy` (79), `LogFiles` (17), `WDI` (16) |
| Taken by WAC alone | 10,679: executables whose signature does not hold (5,300 of them copies the test bench left in `C:\wactest`), signature catalogs (1,205), event logs and their providers' resource files, Defender (98) |

The 140 common exhibits with a different fingerprint are files Windows keeps rewriting — event logs (71), Prefetch (38), hives (6), SRUM (3): both runs start from the same state, but Windows changed them since start-up, differently each time. Their creation date is identical in 1,146 cases out of 1,151. KAPE's own coverage is the one backlog items B-016, B-031 and B-032 already follow.

## 4.2 What is recorded of each exhibit

| | WAC (manifest) | KAPE (copy log) |
|---|---|---|
| Fingerprints | MD5, SHA-1, SHA-256 | SHA-1 |
| File dates | the 4 of `$STANDARD_INFORMATION` and the 4 of `$FILE_NAME`, to the 100 ns, in UTC and in the suspect's local time | 3 of `$STANDARD_INFORMATION` (creation, modification, access), to the 100 ns, in UTC, without the MFT record change |
| Location | MFT record number | path alone |

| | WAC (manifest) | KAPE (copy log) |
|---|---|---|
| Method | stated for every exhibit (raw NTFS reading, live observation) | ordinary copy; “deferred” raw reading for locked files (181 out of 2,292) |
| Failure | code and cause in the manifest, exhibit by exhibit | console and log |
| Integrity | sealed manifest (SHA-256), checked again at every conversion | — |

> **Key point**
>
> The `$FILE_NAME` dates are those an intruder does not rewrite with the ordinary functions: a backdated file (“timestomping”) keeps its real creation date in `$FILE_NAME`. KAPE’s log does not have them; WAC’s manifest gives them for every exhibit, with the MFT record that finds it again in `mft.json`.

CHAPTER 5

# Traces left on the examined machine

## 5.1 The measure

A clone of the test machine, without network, is frozen with Sysmon installed; the tools are on a USB key, drive K:. Every run starts from that same state: the system disk and the key are fresh layers over the frozen images. Once started up, each tool is launched the same way — a scheduled task of the user, with its elevated rights, as an examiner from an elevated prompt.

**Files**
: The USN of C:'s change journal is read just before and just after the tool: every file created, changed, renamed or deleted between the two is in it, even one created then deleted during the run. The readings are written to the key, after the end bound.

**Author**
: Sysmon gives the tool's process tree and what each process wrote, files and registry. A write of another process that names the tool (Amcache, Prefetch, BAM) is Windows reacting to its start.

**Control**
: A run that starts nothing (`ver`) measures the launcher and Windows' own activity; what it shows too is not laid on the tool.

**Deferred**
: The ShimCache is written at shutdown only: every run ends with a clean shutdown, a restart and its reading.

## 5.2 What each one leaves

| Tool | Files written on C: | Registry written | Windows' reactions |
|---|---|---|---|
| WAC (collection) | none | none | Amcache: one entry during the run; ShimCache: one entry at shutdown |

| Tool | Files written on C: | Registry written | Windows' reactions |
|---|---|---|---|
| KAPE, targets | none | 4 Internet Settings `ZoneMap` values in the user's hive; 169 open-or-create calls (certificate stores, policies) | Amcache and ShimCache for `kape.exe` |
| KAPE, targets and modules | 28: 14 in `ProgramData\Microsoft\Windows\Internet Explorer\`, 14 in the user's Temp (`system-commandline-utf8.tmp`) | the same 4 values, rewritten by 14 of the 15 processes (136 writes); about 2,600 open-or-create calls | Amcache and ShimCache for KAPE and 14 tools |
| EZ tools, live | none | no value; 54 open-or-create calls (policies, `Tcpip`, `Internet Settings`) | Amcache and ShimCache for the 6 tools |

No process of any tool made Windows write an event in its logs: those of the measure window are the services' logons, Defender, group policy, common to the control run or unrelated to the tool. All of them read the volumes raw, which Sysmon records (`RawAccessRead`) and an EDR may report.

**Open-or-create**
`RegCreateKeyEx` creates the key if it is missing and opens it otherwise; Sysmon does not tell the two apart. The .NET libraries open the certificate stores and the policy keys that way. A created key changes date; a key merely opened does not.

**The `ZoneMap` values**
Written by WinINet's start-up in every .NET process that loads it, they change the examined user's `NTUSER.DAT` hive, hence its log and the key's date.

**The .NET files**
The runtime leaves one "breadcrumb" per assembly loaded in `BreadcrumbStore`, and the System.CommandLine library a file per tool in the user's Temp. Started alone, the EZ tools left none; started by KAPE, they did.

**Prefetch and BAM**
None of the 22 programs started from the USB key got a Prefetch file or a BAM value, while `cmd.exe`, started from the system disk by the same task, has its BAM value. A tool copied onto the machine and started from there would get them.

**The .NET prerequisite**
KAPE and the EZ tools need the .NET 9 runtime, present on the clone. On a machine without it, installing it writes hundreds of files and keys; WAC is a self-contained executable.

## 5.3 What the measure fixed in WAC

WAC cleared the screen at start-up with `system("cls")`, which starts `cmd.exe`: a process created on the examined machine and the Prefetch of `CMD.EXE` updated, at every collection. The screen is now cleared through the console API, WAC imports no function able to start a process any more, and the build script refuses a version that would import one.

> **Key point**
>
> Every trace of WAC is explained: it starts no process, writes no file on the machine, no registry value, no event. What remains is what Windows leaves of any program started — an Amcache entry, a ShimCache entry at shutdown —, unavoidable and described in the harmlessness file, and the raw reading of the volumes, visible to an EDR.

CHAPTER 6

# Accuracy

## 6.1 In favour of WAC

**Hive logs replayed in order**
The `.LOG1` and `.LOG2` logs keep, behind the recent entries, leftovers of an earlier generation. RECmd applies them in file order, hence after the newer ones. It thus rolls SYSTEM, SECURITY, DEFAULT and a service profile's hive back to an older state: in SYSTEM, sequence 0x3618 instead of 0x3620, and four last BAM executions shown 13 minutes early. WAC sorts the entries by sequence and stops at the break in the chain. The logs themselves confirm it, Marvin32 digests included.

**Events of cleared logs**
A cleared log keeps its records: WAC reads them again and flags them `EvtRecovered`, never presenting them as live; EvtxECmd does not see them.

**Event values as Windows renders them**
Hexadecimal `0x3e7`, GUIDs in braces, booleans `true`, binary `69060000`, dates with their `Z`, U+200E marks kept: the forms Event Viewer shows and one searches for. EvtxECmd changes eight of them, and is wrong on six values, such as a boolean `False` that Windows renders `true`.

**Nothing invented**
A value absent from the source is not emitted. AmcacheParser writes 2,340 default values absent from the hive (`IsPeFile: False` for a PE executable), SrumECmd 7,086 (0 or $-1$).

**Partial Amcache SHA-1 flagged**
For a file larger than 30 MiB, Amcache hashes only the first 31,457,280 bytes (measured on `mrt.exe`). WAC says so (`Sha1CoversBytes`); without it, comparing with the whole file's SHA-1 would wrongly conclude to a modification.

**No misleading field**
The ShimCache "executed" flag, which AppCompatCacheParser gives, says "no" for 105 of the 117 binaries that Prefetch proves were run. WAC does not collect it: that "no" would lead to conclude a program never ran.

**Precision to 100 ns**
WAC gives every date at its source's precision. By default, PECmd, LECmd and SrumECmd round to the second, and LECmd ignores `--mp` in its CSV.

**Dated local time**
Every date is given in UTC and in the suspect's local time, with the offset in force at that date. The EZ tools give UTC only, without saying so by a `Z`.

**Others' readings checked**
SBECmd's "FirstInteracted" and "LastInteracted", RECmd's "CreatedOn" of an account, are

readings of key write times. WAC publishes those times themselves: anyone can draw their own reading from them, without inheriting a ready-made one. SBECmd also gives a wrong `NodeSlot` (0) to first-level folders.

**SrumECmd's ESE reading errors**
17 energy rows out of 887 carry the content of another row or an identifier absent from the table; dissect.esedb sides with WAC.

**Collection**
Each WAC exhibit carries MD5, SHA-1 and SHA-256, the eight NTFS dates of its original and its MFT entry, in a sealed manifest. KAPE gives one SHA-1 and three dates.

## 6.2 In favour of the EZ tools

**Registry coverage**
RECmd's Kroll batch reads keys WAC ignores: installed programs, Defender exclusions, RecentDocs, MountPoints2, known networks, the NTFS last-access update setting, shutdown time, among others. Item B-031.

**File coverage**
Against WAC, KapeTriage now only copies the browsers and the Windows timeline, planned (B-016, B-010), and the `desktop.ini` files, left out. The shortcuts outside Recent, the Default profile's hive, `$Secure:$SDS`, `$Boot`, `$UsnJrnl:$Max`, `$TxfLog`, Defender's WPP traces and the DPAPI master keys, which it alone took, are collected and converted since item B-032.

**Speed on small artefacts and in collection**
See the previous chapter.

**Ready-to-read interpretations**
The EZ tools split and name some fields: executable, date and description of a SRUM application, known folder names, a file's extension. WAC gives the raw value, as Windows stores it.

> **Limit**
> What WAC does not collect, it cannot weigh: KAPE's coverage remains wider on browsers and several registry keys, until B-016, B-031 and B-032 are done.

CHAPTER 7

# Output formats

## 7.1 What each tool writes

| Tool | Formats | Note |
|---|---|---|
| AmcacheParser | CSV | 8 files, one per table |
| AppCompatCacheParser | CSV | |
| RBCmd | CSV | |
| SrumECmd | CSV | 7 files |
| EvtxECmd | CSV, JSON, XML | |
| JLECmd | CSV, JSON, HTML | |
| LECmd | CSV, JSON, HTML, XML | |
| MFTECmd | CSV, JSON, bodyfile | |
| PECmd | CSV, JSON, HTML | |
| RECmd | CSV, JSON | plus one CSV per plugin |
| SBECmd | CSV, JSON | |
| KAPE | copy of the files, CSV copy log | |
| WAC | JSON for every artefact | one file per artefact, plus the sealed manifest |

## 7.2 What sets them apart

Four tools out of eleven write CSV only. In the others, the JSON transcribes the CSV line: one text line per record (JSON Lines), flat.

**Types**
In the EZ tools, numbers are strings: a run count is written `"RunCount":"458"`, a record number `"EventRecordId":"21572"`. WAC writes numbers, booleans, strings.

**Structures**
In the EZ tools, a list becomes a comma-joined string (the files a Prefetch loaded, its directories), and an event's data are JSON escaped *inside* a string (`Payload`), to decode a second time. WAC nests: a Prefetch's volumes and their directories, a shortcut's items, an event's `Name`/`Value` pairs.

**Dates**
In the EZ tools, UTC is not flagged (`2026-09-30 23:04:46`) and precision depends on an option. WAC gives full ISO 8601, in UTC (…`Z`) and in local time with its offset (…`+02:00`).

**Missing value**
The EZ tools write a default value, or an empty cell. WAC omits the field; an artefact not collected is declared so (`NotRequested`, `NotCollected`), so that an empty list is not taken for an absence of traces.

**Provenance**
WAC's manifest ties every value to the exhibit it comes from, with its hashes and original dates.

**Console**
The EZ tools detail their work on the console; some write the decoding itself there when no file is asked for. WAC writes only its progress there: everything it finds is in the files.

> **Key point**
> For an analyst, the difference shows at the first processing: a WAC JSON loads as it is into an analysis tool or into WAE, with nothing to split again; an EZ output is handled like a spreadsheet, whose numbers must be converted back, lists split again and event data decoded again.

CHAPTER 8

# What the confrontation taught WAC

Each defect below produced a valid JSON that was wrong, or incomplete. All are fixed, with a check that would have caught them.

| Artefact | Defect fixed |
|---|---|
| $MFT | stale extension records merged with the base |
| EVTX | floats cut to 6 digits; binary as “69 06 00 00”; hexadecimal integers and GUIDs in the wrong form; ProcessingErrorData ignored (21 events without data); events of cleared logs not read |
| Shortcuts | URI glued to the block after it; reading 2 bytes off without an ID list; ExtraData blocks not read (machine, tracking); target size not read; null DROIDs emitted |
| Jump lists | open count and MRU rank not read; DestList format version 1 not supported; application name corrupted |
| Shellbags | folder lost with everything below it; the folder’s key time, rank and NodeSlot missing; “Home” folder without a name |
| Amcache | 2 tables out of 14 and 4 to 6 values read: no SHA-1, size, driver or device |
| ShimCache | tabs of packaged applications replaced by spaces |
| SRUM | FILETIME dates left as numbers; UTF-16 text read as ANSI |
| Registry | BAM limited to accounts with a profile (a deleted account lost its executions); UserAssist foreground time not read |
| Start-up | `system("cls")` started `cmd.exe` on the examined machine (traces measure) |
| Test harness | a crashing check silently stopped every following one |

CHAPTER 9

# Limits of the comparison

- One machine, virtual, running Windows 11: some artefacts are few there (3 shortcuts in Recent, 2 USB keys). The formats are the same on a real machine, but not every rare case is represented.
- The two collections follow each other instead of being simultaneous: the files written between them ($MFT, logs, Prefetch) rightly differ.
- The EZ tools ran with their maximum precision options; by default, they lose more.
- The traces are measured on a clone without network, one run per tool: a write of Windows the control run did not show is not laid on the tool unless Sysmon gives it to one of its processes; it stays listed as not attributed in `footprint-diff.py`'s results.
- Process creation auditing (Security 4688) is not active on the test machine: where it is, every tool leaves an event per process — one for WAC, fifteen for KAPE and its modules.
- The comparison covers the common artefacts. What only one tool reads has no reference here: its accuracy rests on the checks of the validation report.